

I will structure this as a **layered product plan**, then detail it **one layer at a time** so you can save each section cleanly.

I'll start with:

- Global architecture map

Layer list

Bundle placement

Layer 1 detailed plan

Then we can continue with **Layer 2** in the next message.

Global architecture direction

The system should be built as a stack, not as disconnected setup scripts:

- config

prepares machines, roles, users, mounts, package policy, Python environments, Runpod workspaces, publishing paths, and safe runtime checks
- Agentfield

owns experiment intent, controller flow, agent selection, execution state, status, result tracking, and later campaign-level experiment automation
- Paperclip-Agentfield adapter

connects Paperclip jobs/actions to Agentfield experiments and returns status, metrics, artifacts, and failure reasons
- Paperclip

gives the human-facing dashboard, inbox, governance, job overview, review loop, and end-to-end visibility

That matches your Agentfield notes: config prepares users, environments, mounts, and packages; Agentfield becomes the experiment-aware layer using `GRNExperiment` , controller status/results, an agent registry, a reasoner invoker, and GRN agents. The Paperclip notes add the missing bridge: Paperclip should stay the UI/control surface, while an adapter translates Paperclip jobs into Agentfield runs and returns status/results.

Layer grouping

I would group the real-world work into **five layers**.

- Layer 1 – Runtime substrate

1. Runpod portable runtime base

7. Remote model brain endpoint
- Layer 2 – Role workstations

2. Research Scientist GRN workspace

6. AI Engineer agent/platform dev environment

9. Obsidian writing machine

10. Publisher LaTeX/paper export
- Layer 3 – Research execution loops

3. NCA-ART research stack

4. Parameter search comparison tools

5. Runpod training/inference loop
- Layer 4 – Knowledge, reasoning, and writing automation

8. OpenClaw + PKM reasoning workspace
- Layer 5 – Platform orchestration

11. Agentfield experiment runtime foundation

12. Paperclip-Agentfield adapter

13. Agentic GRN discovery platform

Some items you listed are not standalone bundles; they are **concretizations inside bundles**. For example, `prepare_docker_runtime_policy` , `check_docker_gpu_access` , `check_kubernetes_context` , and `prepare_remote_compute_profile` belong partly to **Runpod portable runtime base** and partly to **AI Engineer agent/platform dev environment**. I would not make them a fourteenth bundle unless the remote compute story becomes large enough later.

Layer 1 — Runtime substrate

Product-owner view

Product goal: make the machine and remote model substrate reliable before any science automation is built.

This layer answers:

- Can I start from a clean local or Runpod runtime and know where repos, environments, datasets, models, checkpoints, and run outputs should live?
- Can I verify GPU, CUDA, PyTorch, Docker, Terraform, Kubernetes, and model endpoint readiness without accidentally launching expensive jobs?
- Can local code call a remote model through one stable contract?

Why this matters: without this layer, every later research workflow becomes fragile. You risk reinstalling packages manually on Runpod, losing outputs, using the wrong Python environment, or mixing one-off runtime state with reproducible experiment policy. Your Runpod plan explicitly points toward Docker as the stable base, config as the role/environment policy layer after pod start, and persistent storage for repos, environments, data, runs, models, and checkpoints.

Layer outcome: after Layer 1, you can bring up a Runpod/local runtime, inspect the workspace, verify GPU/model readiness, and call a remote model from local code through a stable API.

Bundle 1 — Runpod portable runtime base

Scrum-master view

Bundle objective: create a repeatable Runpod/local runtime foundation that prepares paths, checks hardware/software readiness, and avoids manual reinstall drift.

This bundle should not run experiments yet. It should make the runtime safe and inspectable.

User story

- As the operator,
I want config to prepare and verify a Runpod-compatible workspace,
so that GRN research, model training, inference, and Agentfield work can run from
known paths without rebuilding the machine by hand every time.

Concretizations / future managed steps

- prepare_runpod_workspace
- check_runpod_workspace
- check_gpu_runtime
- check_cuda_torch_runtime
- prepare_nca_art_workspace
- prepare_runpod_volume_layout
- prepare_docker_runtime_policy
- check_docker_gpu_access
- prepare_remote_compute_profile

Optional later, when the platform layer becomes real:

- prepare_terraform_runtime_policy
- check_kubernetes_context

I would keep Terraform and Kubernetes as checks/policy at first, not active provisioning.

What each step should do

prepare_runpod_volume_layout

Creates or verifies a persistent workspace layout like:

- /workspace
 - /repos
 - /envs
 - /data
 - /runs
 - /models
 - /checkpoints

What this does for the work: it gives every later workflow a stable place to put code, environments, candidate graphs, outputs, model caches, checkpoints, and experiment runs.

prepare_runpod_workspace

Creates the expected project-level directories and safe placeholders.

Likely paths:

```
/workspace/repos/agentfield
/workspace/repos/nca-art
/workspace/envs
/workspace/data/candidates
/workspace/runs
/workspace/models
/workspace/checkpoints
```

What this does for the work: it makes a fresh pod usable without guessing where files should go.

check_runpod_workspace

Non-mutating check.

It should report:

```
runtime mode: local / runpod / unknown
workspace root exists
persistent volume looks mounted
repos path exists
envs path exists
runs path exists
models path exists
checkpoints path exists
```

What this does for the work: it lets you confirm the pod is ready before spending GPU time.

check_gpu_runtime

Non-mutating check.

Should inspect:

```
nvidia-smi available?
GPU visible?
driver reported?
CUDA visible?
```

It must not start training.

What this does for the work: it prevents starting a training run on a pod that cannot see the GPU.

check_cuda_torch_runtime

Checks the target Python environment’s PyTorch/CUDA state.

Should answer:

```
Which target user?
Which Python environment?
Can Python import torch?
Does torch see CUDA?
Which GPU name?
```

What this does for the work: it confirms that the role environment, not just the machine, can use CUDA.

prepare_nca_art_workspace

Creates the empty workspace skeleton for the future NCA-ART project.

Possible paths:

```
/workspace/repos/nca-art
/workspace/data/candidates
/workspace/runs/nca-art
/workspace/checkpoints/nca-art
```

What this does for the work: it creates the landing zone for the first GRN/NCA/ART prototype without yet implementing the science code.

prepare_docker_runtime_policy

Creates policy/config files describing what belongs in Docker versus what belongs in config.

The policy should reflect your Runpod notes:

Docker image:

- OS packages
- CUDA base
- Python/micromamba base
- common utilities
- optionally stable PyTorch base

config:

- users and roles
- Python environment policy
- role packages
- workspace paths
- experiment-specific settings

persistent volume:

- repos
- envs
- data
- runs
- models
- checkpoints

never bake into Docker:

- secrets
- private data
- unpublished manuscript content
- one-off experiment outputs

What this does for the work: it stops the project from becoming a huge brittle Docker image or a manual reinstall habit.

check_docker_gpu_access

Non-mutating check.

It should inspect Docker availability and whether GPU-aware container execution appears possible. It should not pull images or start containers unless explicitly designed later as a separate active step.

What this does for the work: it tells you whether a Docker-based Runpod/local runtime path is realistic on the current machine.

prepare_remote_compute_profile

Creates a machine profile policy describing local, Runpod, or future cluster mode.

Possible file later:

/home/vmuser/.local/etc/config-sh/runtime/compute-profiles.tsv

Example profile values:

local
runpod-pod
runpod-serverless
kubernetes-dev

What this does for the work: it lets config choose safe defaults for paths and checks depending on where you are working.

Files likely involved

/home/vmuser/.local/etc/config-sh/runtime/runpod.env
/home/vmuser/.local/etc/config-sh/runtime/workspace-paths.env
/home/vmuser/.local/etc/config-sh/runtime/compute-profiles.tsv
/home/vmuser/.local/etc/config-sh/bootstrap/steps.tsv
/home/vmuser/.local/etc/config-sh/bootstrap/profiles/research-scientist.plan
/home/vmuser/.local/etc/config-sh/bootstrap/profiles/ai-engineer.plan
/home/vmuser/.local/lib/config-sh/installers.sh
/home/vmuser/.local/bin/config.sh

Acceptance criteria

config --target researchscientist config-show
shows runtime/workspace policy without secrets

```
sudo config --target researchscientist bootstrap step prepare_runpod_volume_layout
  creates/verifies workspace directories only

sudo config --target researchscientist bootstrap step check_runpod_workspace
  reports workspace state without modifying experiments

sudo config --target researchscientist bootstrap step check_gpu_runtime
  reports GPU visibility without starting training

sudo config --target researchscientist bootstrap step check_cuda_torch_runtime
  checks target Python CUDA readiness

No training is started.
No Runpod job is launched.
No Docker image is built.
No Kubernetes object is applied.
No private data or secrets are printed.
```

Postcheck focus

```
bash -n config.sh
bash -n installers.sh
config bootstrap steps | grep runpod
config bootstrap steps | grep gpu
config bootstrap steps | grep cuda
sudo config --target researchscientist bootstrap step check_runpod_workspace
sudo config --target researchscientist bootstrap step check_gpu_runtime
```

Bundle 7 — Remote model brain endpoint

Scrum-master view

Bundle objective: establish the first reliable remote model loop:

```
local code -> remote model -> response
```

This should stay minimal. Your research assistant plan says the goal is not to build the full system immediately; the first goal is one reliable loop where local code calls a Runpod-hosted model through a standard chat API.

User story

```
As the AI engineer,
I want a stable local client and router for a Runpod-hosted model,
so that OpenCode, OpenClaw, Agentfield reasoners, and future agents can all use
the same remote model contract without each tool inventing its own connection.
```

Concretizations / future managed steps

```
prepare_remote_model_client
check_runpod_brain_endpoint
prepare_brain_router_project
check_opencode_remote_model_config
```

The phrase:

```
local code -> remote model -> response
```

is not a step name. It is the acceptance loop for this bundle.

What each step should do

prepare_remote_model_client

Creates or verifies a small project containing:

```
.env.example
runpod_brain_client.py
brain_router.py
prompts.py
```

```
smoke_test.py
requirements.txt
README-minimal.txt or inline help only, depending project style
```

Since you previously restricted guide creation inside config-tool closure work, for this real-world bundle we should decide whether this creates a project README inside the actual remote-model starter project. That is not the same as generating public guides; it is part of the project scaffold.

What this does for the work: it gives all future agents one small stable way to call the remote model.

check_runpod_brain_endpoint

Non-mutating check.

It should verify:

```
RUNPOD_API_KEY is set
RUNPOD_ENDPOINT_ID is set
base URL can be formed
optional dry HTTP health/chat smoke test if explicitly enabled
```

Default should avoid spending tokens unless a smoke flag is passed.

What this does for the work: it confirms the model endpoint is reachable before OpenClaw, Agentfield, or Paperclip tries to depend on it.

prepare_brain_router_project

Creates the local routing layer:

```
execute(task)
analyze(task)
```

Later this can expand to:

```
plan(task)
summarize(task)
trriage_failure(task)
rank_hypothesis(task)
draft_section(task)
```

But the first version should stay small.

What this does for the work: it stops your application code from calling the model directly everywhere. Agents use the router, not the raw endpoint.

check_opencode_remote_model_config

Checks that OpenCode or the editor-side model config points to the same endpoint/model alias.

What this does for the work: it makes the coding assistant and application runtime use the same model backend when desired.

Files likely involved

```
/home/vmuser/.local/etc/config-sh/runtime/runpod.env
/home/vmuser/.local/etc/config-sh/runtime/remote-model.env
/workspace/repos/research-assistant
/workspace/repos/research-assistant/.env.example
/workspace/repos/research-assistant/runpod_brain_client.py
/workspace/repos/research-assistant/brain_router.py
/workspace/repos/research-assistant/prompts.py
/workspace/repos/research-assistant/smoke_test.py
/workspace/repos/research-assistant/requirements.txt
```

Acceptance criteria

```
prepare_remote_model_client
  creates a minimal remote model client project or verifies it already exists

check_runpod_brain_endpoint
  can confirm endpoint configuration without exposing the API key

prepare_brain_router_project
  creates execute/analyze routing functions

check_opencode_remote_model_config
  reports whether editor model config matches the Runpod endpoint pattern
```



```
The smoke test can prove:
  local code -> remote model -> response

No model is changed.
No endpoint is created automatically.
No key is printed.
No large inference job is run by default.
```

Postcheck focus

```
bash -n config.sh
bash -n installers.sh
config bootstrap steps | grep remote_model
config bootstrap steps | grep brain
sudo config --target aiengineer bootstrap step prepare_remote_model_client
sudo config --target aiengineer bootstrap step check_runpod_brain_endpoint
```

If a real smoke test is allowed:

```
su - aiengineer
cd /workspace/repos/research-assistant
python smoke_test.py
```

But that should be an explicit manual action because it may spend endpoint tokens or compute.

Layer 1 summary

Layer 1 concretises **runtime trust**:

```
Runpod workspace exists.
Persistent paths are known.
GPU/CUDA can be inspected.
Docker/runtime policy is clear.
Remote model endpoint has one stable client.
Local code can call remote model through a thin contract.
```

This gives the rest of the platform a base to stand on. The next layer should be **Layer 2 — Role workstations**, where we map Research Scientist, AI Engineer, Publisher, Obsidian, and LaTeX into concrete setup bundles.